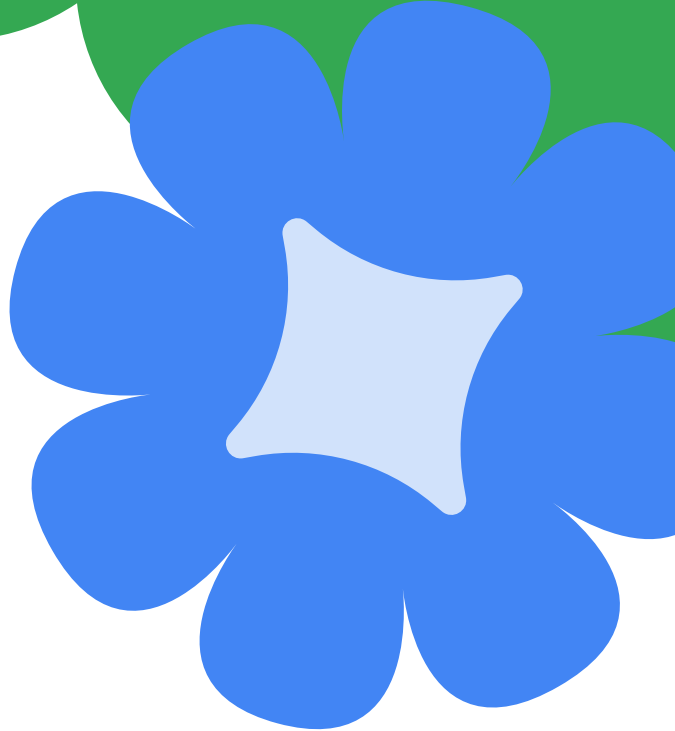




Understand your setup



Before you begin, ensure you have:

Language support

Agent Development Kit (ADK) supports multiple programming languages:

- Python 3.11+ (covered in this course)
- Java 17+ (see [Java Quickstart](#))

This course uses Python for examples. The concepts apply to all supported languages.

Assumptions

This guide assumes:

- You have administrator/sudo access to install packages
- You're familiar with basic terminal commands
- You can create and edit text files
- You have internet connectivity for package downloads

Note: All commands in this guide show variants for different operating systems where they differ.

Required software (Python)

- Python 3.11 or higher – ADK requires Python 3.11+
 - Check your version: `python --version` or `python3 --version`
 - If needed, download from python.org
- **pip** – Python package installer (included with Python)
 - Verify: `pip --version` or `pip3 --version`
- **Terminal/command prompt** – Access to a command-line interface
 - Mac: Terminal app
 - Windows: PowerShell or Command Prompt
 - Linux: Your preferred terminal

System compatibility

- ✓ macOS
- ✓ Windows 10/11
- ✓ Linux (Ubuntu, Debian, Fedora, etc.)

Step-by-step setup

Follow these steps in order to set up your ADK development environment.

Step 1: Verify Python installation

First, check that you have Python 3.11 or higher installed.

Check Python version:

```
Shell  
python --version
```

or

```
Shell  
python3 --version
```

Expected output: **Python 3.11.x** or higher (for example, **Python 3.12.0**)

If Python is not installed or the version is too old:

- Download from python.org
- Install Python 3.11 or newer
- Restart your terminal after installation
- Re-run the version check



Step 2: Create a workspace directory

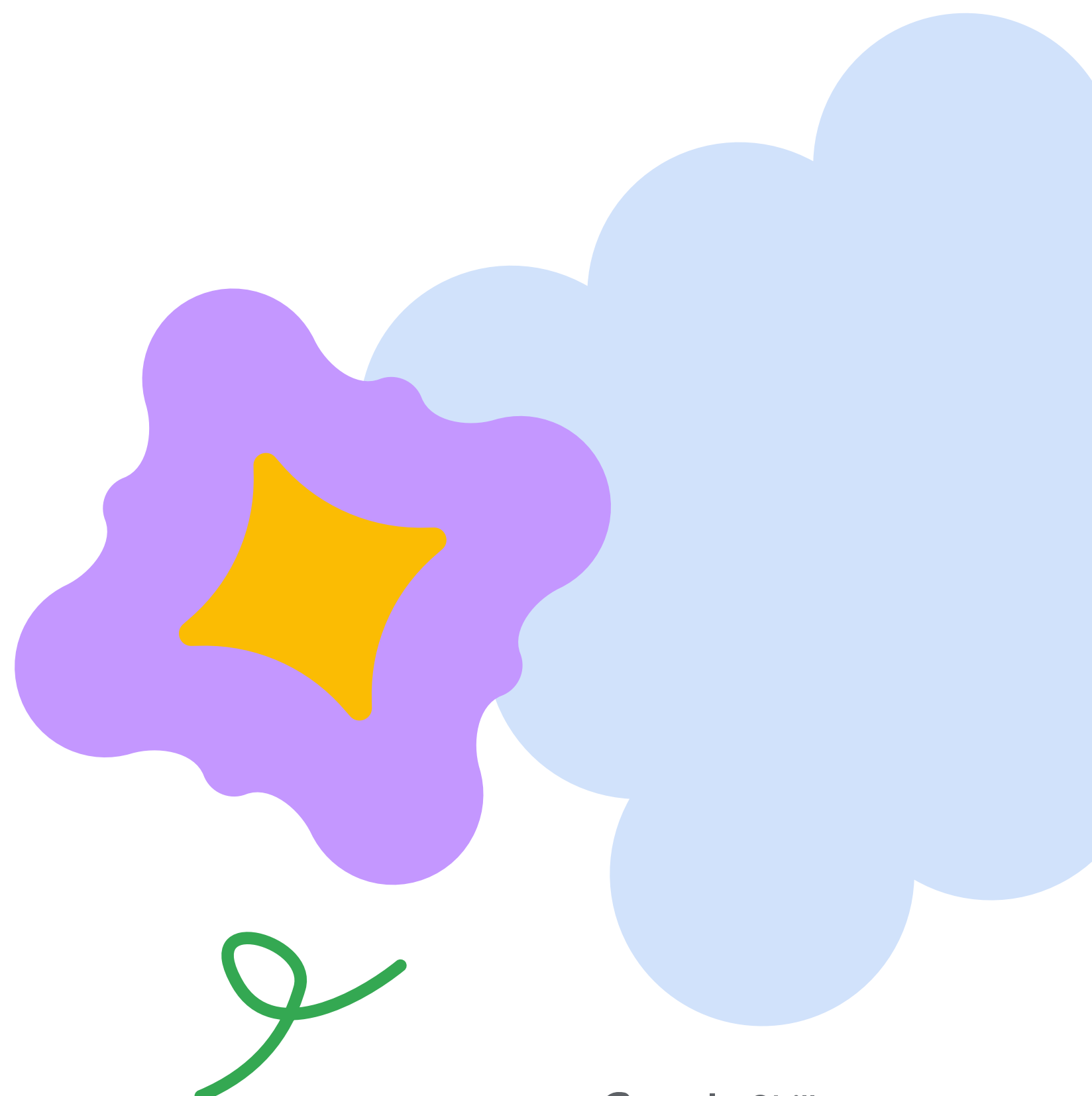
Create a dedicated folder for your ADK projects:

```
Shell  
mkdir adk-workspace  
cd adk-workspace
```

What this does:

- Creates a new directory called **adk-workspace**
- Changes into that directory
- Keeps all your agent projects organized in one place

Windows users: These commands work in both PowerShell and Command Prompt.



Step 3: Create a virtual environment

A virtual environment isolates your project dependencies from your system Python installation, preventing conflicts between different projects.

Create the virtual environment:

```
Shell
python -m venv .venv
```

If `python` doesn't work, try:

```
Shell
python3 -m venv .venv
```

What this does:

- Creates a folder called `.venv` containing an isolated Python environment
- This environment has its own Python interpreter and package space

Activate the virtual environment:

On Mac/Linux:

```
Shell
source .venv/bin/activate
```

On Windows (PowerShell):

```
None
.venv\Scripts\Activate.ps1
```

On Windows (Command Prompt):

```
None
.venv\Scripts\activate.bat
```

How to tell it's activated:

- Your terminal prompt should now show `(.venv)` at the beginning
- Example:
`(.venv) user@machine:~/adk-workspace$`

Important: You must activate this virtual environment every time you open a new terminal to work on your agent projects.

Step 4: Install ADK

With your virtual environment activated, install the ADK:

```
Shell
pip install google-adk
```

What this installs:

- The ADK framework with agent abstractions (`LlmAgent`, `Runner`, etc.)
- CLI tools (adk command)
- Dependencies for working with Google's Gemini models

Verify the installation:

```
Shell
adk --version
```

Expected output:

Version number like `1.0.0` or higher

If `adk --version` fails:

- Check that your virtual environment is activated (you should see `(.venv)` in prompt)
- Try deactivating and reactivating: `deactivate` then re-activate
- Reinstall: `pip install --force-reinstall google-adk`

Step 5: Get your API key

Your agent needs access to a large language model or LLM (the “Model” component from course 1). For this course, we’ll use Google’s Gemini models.

Here you have two options:

Option A:

Gemini API via Google AI Studio
(recommended for learning)

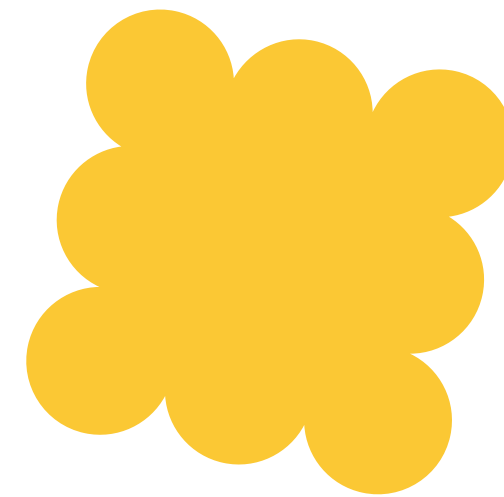
This is the simplest option for getting started.

Steps:

- 1 Visit [Google AI Studio](#)
- 2 Sign in with your Google account
- 3 Click **Create API Key**
- 4 Copy the API key (it looks like [AIzaSyC...](#))

Benefits:

- ✓ Quick setup (just needs API key)
- ✓ Free tier available
- ✓ Perfect for development and learning



Option B:

Gemini via Google Cloud Vertex AI
(for production)

Use this option if you need enterprise features or are deploying to production.

Prerequisites:

- Google Cloud account
- Billing enabled (free trial available)
- A Google Cloud project

Steps:

- 1 Create a [Google Cloud project](#) if you don’t have one
 - 2 Enable the [Vertex AI API](#)
 - 3 Install the [gcloud CLI](#)
 - 4 Authenticate
- ```
Shell
gcloud auth application-default login
```
- 5 Note your **Project ID** (visible in Cloud Console)
  - 6 Note your **Location** (for example, [us-central1](#))

#### Benefits:

- ✓ Enterprise-grade features
- ✓ Better scaling and quotas
- ✓ Production-ready security

For this course, we recommend option A (Google AI Studio) for simplicity.



## Step 6: Create your first agent project

ADK provides a command to scaffold a new agent project with the correct structure.

Create the project:

```
Shell
adk create my_first_agent
```

What this creates:

```
None
my_first_agent/
├── agent.py # Main agent code
 (you'll edit this)
├── __init__.py # Python package
 initialization
└── .env # Environment
 variables (you'll edit this next)
```

Understanding the files:

- **agent.py:** This is where you'll define your agent using Python code. This file brings together the model, tools, and orchestration components you learned about in course 1. It contains your `root_agent` definition
- **\_\_init\_\_.py:** Python package initialization file that imports your agent module. This is required for ADK to discover your agent
- **.env:** A special file for storing sensitive information like API keys. ADK automatically loads this file, keeping secrets out of your code

Navigate into the project:

```
Shell
cd my_first_agent
```

Verify the default agent code:

Open `agent.py` in your text editor.  
You should see something like this:

```
Python
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
 model='gemini-2.5-flash',
 name='root_agent',
 description='A helpful assistant
agent.',
 instruction='You are a helpful
assistant.'
)
```

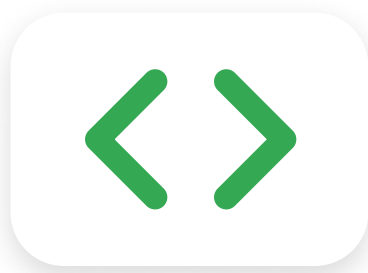
What this code does:

- Imports the `Agent` class from ADK
- Creates a `root_agent` (the main agent ADK looks for)
- Configures it with:
  - **Model:** `gemini-2.5-flash` (the LLM that powers reasoning)
  - **Name:** `root_agent` (required identifier)
  - **Description:** What the agent does (used in multi-agent systems)
  - **Instruction:** How the agent should behave

This is a minimal working agent. In module 2, you'll learn to customize these parameters and add tools.

**Note:** This course uses Python. If you're working with Java, see the [Java Quickstart](#) for Java-specific project structure and setup.

# Step 7: Configure your API key



Now configure the `.env` file with the API key you obtained in step 5.

Open the `.env` file in your text editor.

If using Google AI Studio (option A):

Replace the contents with:

```
Shell
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=your-actual-api-key-here
```

Replace `your-actual-api-key-here` with your actual API key from Google AI Studio.

Example:

```
Shell
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=AizaSyC4lW...
```

If using Vertex AI (option B):

Replace the contents with:

```
Shell
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=us-central1
```

Replace:

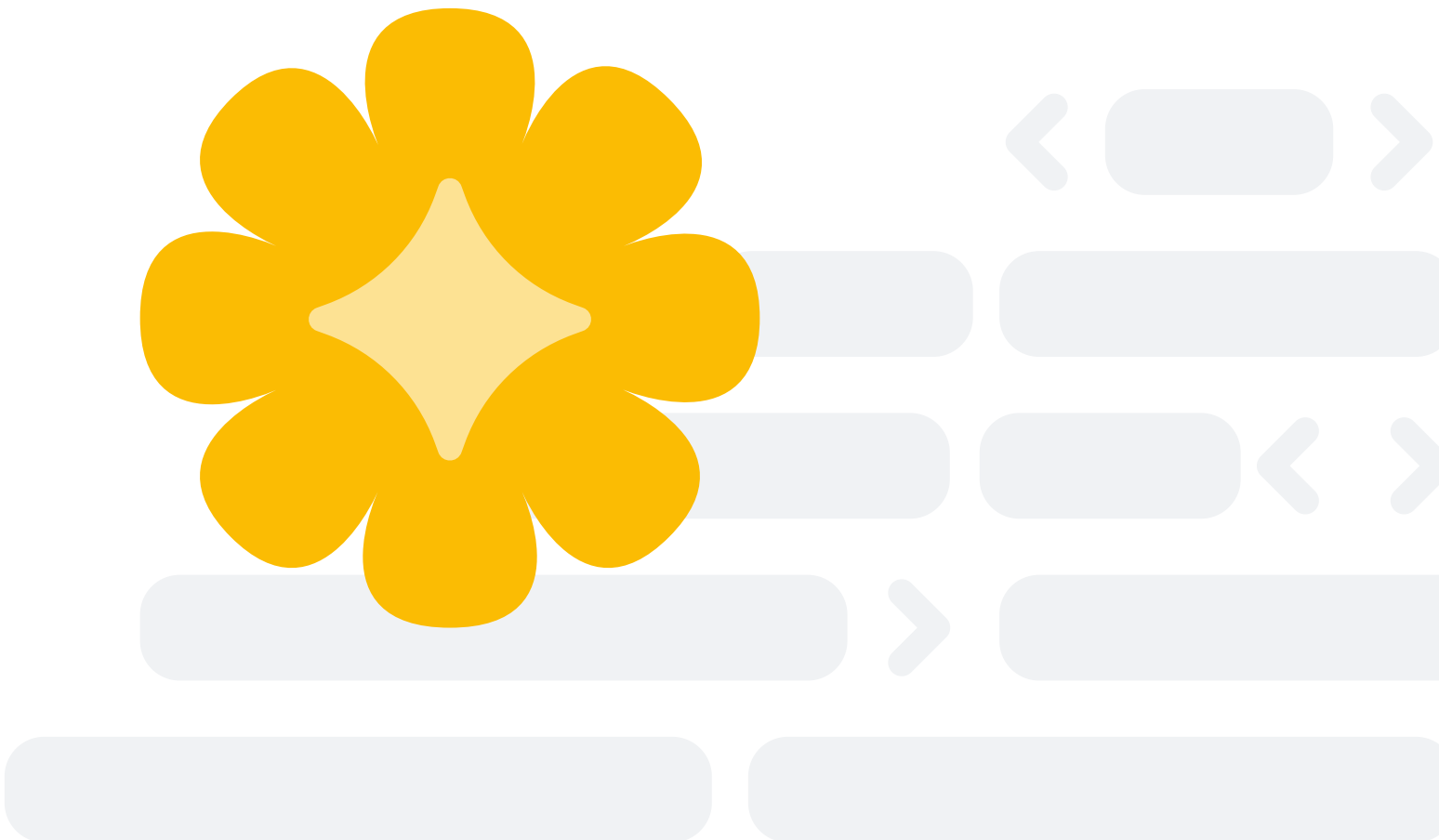
- `your-project-id` with your Google Cloud Project ID
- `us-central1` with your preferred location (if different)

Example:

```
Shell
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=my-agent-project-123
GOOGLE_CLOUD_LOCATION=us-central1
```

Important security note:

- Never commit the `.env` file to version control (Git)
- The `.env` file should be listed in `.gitignore`
- API keys are sensitive credentials. Treat them like passwords





## Step 8: Verify your setup

Let's confirm everything works by running the ADK web interface.

Start the ADK web interface:

```
Shell
adk web
```

Expected output:

```
Shell
INFO: Started server process
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

What this does:

- Starts a local web server at <http://localhost:8000>
- Opens a browser tab automatically
- Provides a visual interface to interact with your agent

Success indicators:

- ✓ No error messages in the terminal
- ✓ Browser opens to the ADK interface
- ✓ You see the web UI with your agent name

If you see errors:

- ✗ “ModuleNotFoundError: No module named 'google.adk'”
  - Your virtual environment isn't activated
  - Fix: Re-activate with [source .venv/bin/activate](#) (Mac/Linux) or [.venv\Scripts\Activate.ps1](#) (Windows)

### ✗ “Invalid API key” or authentication errors

- Check that your .env file has the correct variable names
- For Google AI Studio: Use [GOOGLE\\_API\\_KEY](#) (NOT [GEMINI\\_API\\_KEY](#))
- Ensure no extra spaces around the = sign
- Verify that the API key is correct (copy-paste again from Google AI Studio)

### ✗ “Command 'adk' not found”

- The virtual environment is not activated, or ADK is not installed
- Fix: Activate venv, and then run [pip install google-adk](#)

Stop the web server: Press [Ctrl+C](#) in your terminal to stop the server when you're done testing.



# Understanding your setup

Let's connect what you just built to the concepts from course 1.

## The three components in your environment

Remember: Agent = model + tools + orchestration.

Check Python version:

### 1. Model (configured in step 7)

- Your `.env` file configures access to Gemini
- Gemini is the LLM that provides reasoning and decision-making
- This is the “brain” that understands language and makes choices

### 2. Tools (coming in module 3)

- Functions your agent can call to take actions
- Examples: search the web, read files, and send emails
- These bridge “knowing” to “doing”

### 3. Orchestration (provided by ADK)

- The framework that runs the agent loop
- Manages: Perceive → think → act → check → repeat
- You installed this in step 4 (`pip install google-adk`)

## How the files work together

agent.py – Where everything connects:

You saw the default agent code in step 6. Let's connect it to course 1 concepts:

Shell

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
 model='gemini-2.5-flash', # Model: The reasoning engine (Course 1!)
 name='root_agent', # Identity: Required identifier
 description='A helpful agent.', # Purpose: What this agent does
 instruction='You are helpful.' # Behavior: How to act
 # Tools: You'll add these in Module 3
 # Orchestration: Handled automatically by the Agent class
)
```



Breaking it down:

- **Model ([gemini-2.5-flash](#)):** The LLM that provides reasoning and decision-making
- **Tools (not shown yet):** Functions the agent calls to take actions (module 3)
- **Orchestration:** The Agent class automatically runs the Perceive → Think → Act → Check loop

[.env](#) – Secure credentials:

- Keeps API keys out of code
- Automatically loaded by ADK
- Never committed to Git

[\\_\\_init\\_\\_.py](#) – Package initialization:

- Makes your folder a Python package
- Imports your agent module
- Required for ADK to discover agents

**Note:** ADK also supports YAML-based configuration (see [Agent Config](#)) and Java (see [Java Quickstart](#)). This course focuses on Python code-based agents.

## The development workflow

1. Edit [agent.py](#) – Define your agent's behavior
2. Run [adk web](#) – Test in the web interface
3. Iterate – Make changes, refresh, and test again

Iterate – Make changes, refresh, and test again

- `adk run` – Terminal-based interaction
- `adk api_server` – Deploy as an API service

